

Test 1 — C++ Review & OOP Basics

50-minute exam — ANSWER KEY

part 1 Multiple Choice

1 pt each

- By default, members of a class declared with the `struct` keyword are:
 - ☐ A private
 - ☒ B public
 - ☐ C protected
 - ☐ D decided by the compiler
- An array-based `Stack` only needs one index, `top`, because pushes and pops always happen:
 - ☒ A at the same end
 - ☐ B at opposite ends
 - ☐ C at a random index
 - ☐ D only during construction
- A linked-list `Queue` needs *two* pointers, `front` and `rear`, because it:
 - ☒ A adds and removes at opposite ends
 - ☐ B stores twice the data
 - ☐ C must be sorted
 - ☐ D is required by every linked list
- Given `Widget a(5);` then `Widget b(a);` and `Widget c = a;`, both `b` and `c` are built by the:
 - ☐ A assignment operator
 - ☒ B copy constructor
 - ☐ C default constructor
 - ☐ D destructor
- `operator<<` normally has to be written as a non-member (`friend`) function because:
 - ☒ A the left operand (`cout`) isn't your class
 - ☐ B member functions can't return references
 - ☐ C `friend` is required for any operator
 - ☐ D it runs faster as a free function
- A class owns a raw pointer (`int* data;`) allocated with `new`. Which three members form the “Rule of Three”?
 - ☐ A constructor, destructor, `operator<<`
 - ☒ B copy constructor, destructor, `operator=`
 - ☐ C default constructor, copy constructor, getter
 - ☐ D destructor, `operator+`, comparison operator

Test 1 — C++ Review & OOP Basics

50-minute exam — ANSWER KEY

part 2 Match the Concept

1 pt each

Encapsulation **B**Inheritance **C**Polymorphism **A**

- A. Overloading or overriding methods so behavior adapts to the context it's used in.
- B. Bundling data with the functions that operate on it, and hiding the internal details from outside code.
- C. Organizing classes hierarchically so classes lower in the hierarchy reuse and extend behavior from classes above.

GRADING NOTES**What counts****Part 1** 1 pt per correct letter, no partial credit.**Part 2** 1 pt per correct letter. Watch for students who match by keyword-spotting (e.g. "hierarchical" → C is the giveaway) — that's fine, the point is they know the term.

Test 1 — C++ Review & OOP Basics

50-minute exam — ANSWER KEY

part 3 Find the Bug

model answer — 4 pts

ACCOUNT.CPP

```
class Account {  
    Account(double balance) {    // ← declared before public:  
        this->balance = balance;  
    }  
    double balance;  
public:  
    double getBalance() { return balance; }  
};
```

Does not compile. A class defaults to `private` until a label says otherwise, and the constructor is declared *above* the `public:` label — so it's private. `Account a(500.00);` in `main()` fails with “`Account::Account(double)` is private within this context.” Fix: move the constructor below `public:` (or add an explicit `public:` label above it). This is separate from the second question — making `balance` itself readable from `main()` would additionally require calling `getBalance()`, since `balance` stays private either way.

GRADING NOTES**What counts**

Compiles? Must circle "does not compile." 2 pts for the circle, 2 pts for naming the actual cause (constructor above `public:` → private). "It doesn't compile because `balance` is private" is NOT sufficient — that's a different (true but not asked-about) fact; the failure is on the constructor call, not on `a.balance`.

Test 1 — C++ Review & OOP Basics

50-minute exam — ANSWER KEY

part 4 From Usage to Class

model answer — 10 pts

```
class Song {
private:
    string title;
    string artist;
    int duration;    // seconds -- only known from line 2
    string genre;    // only known from the setGenre() call on line 4

public:
    // Usage showed two different constructors (title+artist+duration
    // on line 2, title+artist only on line 3) -- implementing both,
    // plus the required default constructor:
    Song();
    Song(string, string, int);
    Song(string, string);

    // Getters and setters -- one pair per data member
    string getTitle();
    string getArtist();
    int getDuration();
    string getGenre();
    void setTitle(string);
    void setArtist(string);
    void setDuration(int);
    void setGenre(string);

    // Called on line 5 but never defined for us -- declaration only
    void play();
};
```

GRADING NOTES**What counts**

Part 4 Full credit doesn't require these exact names/types — accept anything self-documenting with matching types (string/int) for title, artist, duration, genre. Must include a default constructor even though line 1 gives no info. A comment (or any indication) noting the two different constructor signatures on lines 2 & 3 is worth partial credit even if only one overload is written — that's the whole point of this question style. `play()` must appear as a declaration; students are not expected to implement it.

Test 1 — C++ Review & OOP Basics

50-minute exam — ANSWER KEY

part 5 Implement the Methods

model answer — 6 pts

```
void Counter::Increment(int amount) {  
    count += amount;  
}  
  
void Counter::Reset() {  
    count = 0;  
}
```

GRADING NOTES**What counts**

Part 5 Must use `Counter::` scope resolution for both. `Increment` must use `+=` (or equivalent) on the existing value, not overwrite it — that's the difference from `Reset`.

Test 1 — C++ Review & OOP Basics

50-minute exam — ANSWER KEY

part 6 From Usage to Class, Round Two

model answer — 8 pts

```

class Employee {
private:
    string name;
    string title;
    double salary;    // only known from line 2

public:
    // Usage showed two different constructors (title+role+salary on
    // line 2, title+role only on line 3) -- implementing both, plus
    // the required default constructor:
    Employee();
    Employee(string, string, double);
    Employee(string, string);

    // Getters and setters -- one pair per data member
    string getName();
    string getTitle();
    double getSalary();
    void setName(string);
    void setTitle(string);
    void setSalary(double);

    // Methods gleaned from usage
    void giveRaise(double);
    void clockIn();
};

```

GRADING NOTES**What counts**

Part 6 Same rubric as Part 4. Accept any self-documenting names/types (string/double) for `name`, `title`, `salary`. Must include a default constructor despite line 1 giving no info, and a comment (or equivalent) noting the differing constructors on lines 2 & 3. `giveRaise(double)` must take a parameter (it changes `salary` by an amount, it's not a plain setter) — that distinction from `setSalary` is worth partial credit on its own. `clockIn()` only needs a declaration.